Advanced Problem-Solving Lab – High-IQ Track

Activity H1 – Solve the "Hidden Problem" Behind the Stated Problem

Title: From Surface Problem to Real Problem

Goal
Train yourself to see the *real* problem, not just the stated one.

Instructions
You are given a "client statement":

"Our exam system is too slow. We want a Python script to speed it up."

1. Identify at least 3 possible *real* problems this statement might hide.
    - Hidden problem 1: _____
    - Hidden problem 2: _____
    - Hidden problem 3: _____
2. For each hidden problem, state:
    - What data would you need to confirm it?
        - Problem 1: _____
        - Problem 2: _____
        - Problem 3: _____
3. Choose ONE hidden problem and re-frame it as a clear technical problem statement (2–3 lines):

New problem statement:

---

4. Design a very small Python experiment that could validate or reject your reframed problem (pseudo-code is fine):

python

 # Experiment idea (what will you measure, on what data, how?)

Activity H2 – Multiple Competing Solutions

Title: Design 3 Solutions Before Coding 1

Goal
Avoid fixation. Explore at least three distinct solution strategies.

Instructions
Problem:

"You receive a stream of log lines (text). You must detect 'suspicious' activity in real time."

1. Give three different solution ideas, each using a distinct main technique or data structure.
    - Approach A (e.g., rule-based / regex / thresholds):

    - Approach B (e.g., statistical / sliding window / counters):

    - Approach C (e.g., simple ML / anomaly detection concept):

---

2. For each approach, list pros and cons (at least one each).
    - Approach A – Pros: _____ Cons: _____
    - Approach B – Pros: _____ Cons: _____
    - Approach C – Pros: _____ Cons: _____
3. Choose one approach to implement first and justify:

I choose approach ___ because

---

4. Sketch high-level Python structure (modules/classes, not full code):

python

 # Modules / classes / key functions list

Activity H3 – Self-Constrained Challenge: “Elegant Code Only”

Title: Solve with Elegance (Clarity, Minimalism, Correctness)

Goal
Produce code that is not only correct, but also minimal and beautiful.

Problem
Implement a mini “pipeline” for processing a list of tasks:

- Each task has an ID, description, and status ("todo", "doing", "done").
- You must:
    - Load tasks from a JSON file.
    - Filter by status.
    - Sort by ID.
    - Print a clean summary.

Constraints (you impose on yourself)

1. Maximum 25 lines of *real* Python (excluding blanks and comments).
2. No function longer than 7 lines.
3. Names must be meaningful and short.

Steps

1. Design your data model in 2–3 lines:

---

2. Decide at least one thing you will *not* do to keep it minimal:

---

3. Write your code here (respecting constraints):

python

 # Elegant pipeline solution

4. Reflection:
    - What did you have to sacrifice (if anything) to keep elegance?

Activity H4 – Complexity and Scaling Thought Experiment

Title: How Will Your Solution Break at Scale?

Goal
Develop realistic intuition about performance and scaling.

Instructions
Take ANY non-trivial function or algorithm you have recently written (search, summarization, scoring, etc.).

1. Describe what it does, in *one line*:

---

2. Estimate its time complexity in Big-O notation, and justify in 1–2 lines:
   - Big-O: O(_____)
   - Because: _____
3. Scaling scenarios – imagine input sizes:
   - $n = 10^3$ (1,000)
   - $n = 10^6$ (1,000,000)
   - $n = 10^9$ (1,000,000,000)

For each, predict: will it still be usable (Y/N) and why?

n	**Usable (Y/N)**	**Reason (1 short sentence)**
10^3		
10^6		
10^9		

4. Suggest one change to improve scalability and briefly describe how it helps:

Activity H5 – Adversarial Thinking: Break Your Own Design

Title: Attack Your Own Program

Goal
Develop robustness and defensive mindset.

Instructions
Choose one of your own projects or mini-tools (file reader, API client, CLI, etc.).

1. Write a short description of what it is supposed to do:

---

2. Now wear the hat of an *attacker* or malicious user.
   List 5 ways to misuse / break it:
   - Attack 1: _____
   - Attack 2: _____
   - Attack 3: _____
   - Attack 4: _____
   - Attack 5: _____
3. For each attack, suggest one defence in plain language:
   - Defence for attack 1: _____
   - Defence for attack 2: _____
   - Defence for attack 3: _____
   - Defence for attack 4: _____
   - Defence for attack 5: _____
4. Implement at least one of these defences in your code and paste the updated snippet:

python

 # Hardened code snippet:

Activity H6 – Cross-Domain Transfer

Title: Apply Known Pattern to a New Domain

Goal
Practice lifting a known idea/pattern from one domain to a different one.

Instructions
Pick a pattern you already know well, e.g.:

- Sliding window
- Two-pointer technique
- Dynamic programming table
- Producer–consumer
- Map–reduce style
- Pipeline of filters

1. Name of pattern: _____
2. Original domain where you learned it (e.g., arrays, strings):

---

3. New domain to apply it in (e.g., logs, time series, learning activities, medical data):

---

4. Describe how you would adapt the pattern to this new domain, in 5–8 lines (can include pseudo-code):

---

5. Optional: implement a small prototype showing this transfer:

python

 # Pattern transfer prototype

Activity H7 – Meta-Cognition: Your Personal Problem-Solving Manual

Title: Write Your Own "How I Solve Problems" Guide

Goal
Turn your tacit skills into an explicit, reusable system.

Instructions

1. List the first three things you usually do when you get a new problem (be honest):
    - Step 1: _____
    - Step 2: _____
    - Step 3: _____
2. Now, design your ideal 6–8 step process (what a "future you" would always do):

1. _____
2. _____
3. _____
4. _____
5. _____
6. _____
7. (optional) _____
8. (optional) _____

3. For each step, write one question you will ask yourself:
    - Step 1 question: _____
    - Step 2 question: _____
    - Step 3 question: _____
    - Step 4 question: _____
    - Step 5 question: _____
    - Step 6 question: _____
4. Final instruction to yourself (one sentence):

"When I feel stuck, I will _____."